

Fase 13 — Fragmentación Horizontal

Materia

BDD

Continuación directa de la Fase 12. Los tres nodos activos (`bdd-nodo01` , `bdd-nodo02` y `bdd-nodo03`) tienen sus snapshots `fase12-completa` tomados. En esta fase se crean dos nuevas máquinas virtuales — `bdd-nodo04` y `bdd-nodo05` — que actuarán como nodos de sharding, y se carga en cada uno el fragmento horizontal que le corresponde según el **Documento de Diseño Distribuido (DDD)** elaborado en la Fase 9. Los datos de `clientes` , `pedidos` y `detalle_pedidos` se distribuyen físicamente entre los dos shards usando el atributo `region` como predicado de fragmentación. Al finalizar, el laboratorio tendrá, por primera vez, dos nodos independientes que almacenan subconjuntos disjuntos de los mismos datos: una fragmentación horizontal real y no una simulación local como el particionamiento nativo de la Fase 12.

A. Objetivos de aprendizaje

Al finalizar esta fase, el estudiante será capaz de:

1. Crear nuevos nodos del laboratorio mediante **clones enlazados** (*linked clones*) de VirtualBox a partir de un snapshot base, aplicando el mismo patrón de aprovisionamiento rápido utilizado en la Fase 11 para `bdd-nodo03` .
2. Configurar la identidad de cada nodo shard (hostname, IP estática, `server_id` de MariaDB) para que coexistan sin conflicto en la red Host-Only.
3. Implementar la **estrategia de carga de fragmentos** usando `mysqldump` `--where` para tablas con atributo de fragmentación explícito y tablas temporales materializadas para fragmentos derivados sin columna propia de partición.
4. Aplicar correctamente la **fragmentación derivada** de `detalle_pedidos` , co-localizando cada línea de detalle con el fragmento del pedido padre al que pertenece.
5. Justificar y demostrar por qué los esquemas de los nodos shard **no incluyen claves foráneas** y cómo la co-localización preserva la integridad referencial por diseño.

6. Verificar las **tres condiciones de correctitud** (completitud, disjunción, reconstrucción) de la fragmentación horizontal sobre los datos reales cargados en nodo04 y nodo05.
 7. Construir la **consulta de reconstrucción UNION ALL** que el coordinador (nodo06) ejecutará en la Fase 14 y validarla manualmente desde nodo01.
 8. Distinguir con precisión la diferencia entre el particionamiento nativo de MariaDB (Fase 12, datos en un solo servidor) y la fragmentación distribuida (Fase 13, datos en servidores físicamente separados).
 9. Cerrar la fase con los snapshots `fase13-completa` en los cinco nodos activos del laboratorio.
-

B. Conceptos teóricos necesarios

1. Diferencia operacional entre particionamiento nativo y fragmentación distribuida.

En la Fase 12 se demostró el particionamiento nativo: las filas se dividen en segmentos físicos *dentro del mismo servidor*. El optimizador de MariaDB conoce todos los segmentos, gestiona las transacciones a través de ellos y puede aplicar poda de particiones en tiempo de ejecución. En esta fase los fragmentos residen en *servidores físicamente distintos*: nodo04 y nodo05 son dos instancias independientes de MariaDB que no comparten disco, memoria ni proceso. La diferencia más importante es que ningún motor puede garantizar atomicidad ni integridad referencial cruzando la frontera de red entre ambos nodos.

Característica	Particionamiento nativo (Fase 12)	Fragmentación distribuida (Fase 13)
Ubicación de datos	Un único servidor	Múltiples servidores físicamente separados
Transparencia	Total: el cliente ignora los segmentos	Requiere coordinador (nodo06, Fase 14)
Claves foráneas	No soportadas entre tablas particionadas	No soportadas entre nodos distintos
Escalabilidad	Vertical (más recursos al mismo servidor)	Horizontal (agregar más nodos)
Objetivo principal	Rendimiento y mantenimiento	Escalabilidad y disponibilidad

2. Fragmentación horizontal primaria y derivada.

La fragmentación horizontal divide las **filas** de una relación en subconjuntos disjuntos, cada uno definido por un predicado de selección:

- *Fragmentación primaria*: el predicado se aplica directamente sobre una columna de la propia tabla. `clientes` y `pedidos` tienen la columna `region`, por lo que los predicados `frag_A: region IN ('norte','este')` y `frag_B: region IN ('sur','oeste')` actúan directamente.
- *Fragmentación derivada*: la tabla hija no tiene columna de fragmentación propia y se fragmenta siguiendo la distribución de su tabla padre a través de la clave foránea. `detalle_pedidos` no tiene columna `region`; se fragmenta derivadamente a través de `pedido_id → pedidos.id`: las líneas cuyo pedido está en `frag_A` van a `nodo04` y las del `frag_B` van a `nodo05`.

3. Co-localización de tablas relacionadas.

Cuando dos tablas se fragmentan bajo el mismo predicado y sus fragmentos se asignan al mismo nodo se dice que están *co-localizadas*. La co-localización es la condición que permite ejecutar JOINS completamente dentro del mismo nodo sin

transferencia de datos por red — la operación más costosa en un sistema distribuido. En este laboratorio:

- `clientes_frag_A` + `pedidos_frag_A` + `detalle_frag_A` → co-localizadas en nodo04.
- `clientes_frag_B` + `pedidos_frag_B` + `detalle_frag_B` → co-localizadas en nodo05.

Esta co-localización permite que la consulta `JOIN clientes → pedidos → detalle_pedidos` se resuelva íntegramente dentro de un único nodo para cada región, sin cruzar la red.

4. Integridad referencial en un sistema distribuido.

MariaDB no puede imponer claves foráneas (`FOREIGN KEY`) entre tablas de servidores distintos. En el sistema centralizado de nodo01, InnoDB garantiza automáticamente que todo `pedido.cliente_id` existe en `clientes.id`. En los shards esta garantía desaparece y la responsabilidad se transfiere a uno de los siguientes mecanismos:

- *Validación en la capa de aplicación*: el código verifica la existencia del cliente antes de insertar un pedido.
- *Scripts de auditoría periódica*: consultas que detectan filas huérfanas de forma programada.
- *Lógica en el coordinador*: nodo06 (Fase 14) puede imponer restricciones lógicas antes de enrutar escrituras a los shards.

En este laboratorio la co-localización de `clientes` y `pedidos` por el mismo predicado garantiza *de hecho* que el cliente referenciado por cada pedido siempre estará en el mismo nodo, preservando la integridad por diseño sin necesidad de FK de motor.

5. Clon enlazado (*linked clone*) en VirtualBox.

Un clon enlazado comparte el disco base del snapshot de origen con la VM original y solo almacena las diferencias (*delta disk*). Ventajas para este laboratorio:

- Se crea en segundos (no copia gigabytes de datos, solo referencia el disco base).
- Ocupa muy poco espacio inicial en disco.
- Se provisiona con el mismo SO, MariaDB y datos de prueba que el nodo fuente.

La contrapartida es que el snapshot de origen no puede eliminarse mientras exista el clon enlazado. Como este laboratorio conserva todos los snapshots, esto es irrelevante en la práctica.

6. Estrategia de carga de fragmentos con `mysqldump`.

`mysqldump --where` exporta solo las filas que cumplan un predicado SQL, generando sentencias `INSERT INTO` exclusivas para ese fragmento. Para tablas con columna de fragmentación explícita (`clientes` , `pedidos`):

Text

```
mysqldump --no-create-info --where="region IN ('norte','este')"  
lab_bdd clientes
```

Para `detalle_pedidos` (fragmentación derivada, sin columna `region`) se crea una tabla temporal en nodo01 que materializa las filas del fragmento mediante un JOIN. Esa tabla temporal se exporta con `mysqldump --no-create-info` y un `sed` sustituye el nombre de la tabla temporal por `detalle_pedidos` antes de importarlo en el shard.

7. Carga completa de `productos` en ambos shards.

La tabla `productos` no se fragmenta horizontalmente en esta fase; lo hará verticalmente en la Fase 14 (dos fragmentos: `V_productos_basico` en nodo04 y `V_productos_detalle` en nodo05). Para que los JOINS entre `detalle_pedidos` y `productos` puedan resolverse localmente en cada shard *antes* de que exista el coordinador, se carga el catálogo completo (10 filas) en ambos nodos. Esta replicación temporal se sustituirá en la Fase 14 por la fragmentación vertical definitiva.

C. Procedimiento paso a paso

Paso 1 — Apagar todos los nodos activos del laboratorio.

La creación de los clones requiere que nodo01 esté detenido para evitar conflictos de IP con los clones que arrancarán temporalmente con la misma dirección.

Paso 2 — Crear `bdd-nodo04` como clon enlazado del snapshot `fase08-completa` de nodo01.

Se elige `fase08-completa` porque es el último snapshot que contiene MariaDB instalado *sin* configuración de replicación — la base más limpia para un nodo shard autónomo.

Paso 3 — Configuración inicial de `bdd-nodo04`.

Arrancar solo nodo04, conectar mediante SSH a la IP heredada del clon (192.168.56.101), y reconfigurar hostname, IP estática (.104), regenerar claves SSH y ajustar MariaDB (`server_id=4`, `bind-address=0.0.0.0`).

Paso 4 — Crear `bdd-nodo05` como clon enlazado del snapshot `fase08-completa` de nodo01.

Mismo proceso que el Paso 2; nodo04 puede seguir corriendo en .104 sin conflicto.

Paso 5 — Configuración inicial de `bdd-nodo05`.

Arrancar nodo05 (hereda .101), conectar vía SSH y reconfigurar hacia .105 con hostname `bdd-nodo05` y `server_id=5`.

Paso 6 — Reiniciar nodo01 y preparar la distribución de datos.

Con los tres nodos corriendo en IPs distintas (.101, .104, .105), crear el usuario de lectura para los shards (`shard_pull`) y materializar las tablas temporales de `detalle_pedidos` para cada fragmento.

Paso 7 — Crear el esquema `lab_bdd` en nodo04 y nodo05.

Esquema sin claves foráneas, con la misma estructura de columnas que nodo01 (incluyendo `ultima_modificacion` añadida en la Fase 10).

Paso 8 — Cargar `frag_A` en nodo04.

Importar `clientes`, `pedidos`, `detalle_pedidos` (fragmento A) y `productos` completo, tirando datos vía `mysqldump` desde nodo04 hacia nodo01.

Paso 9 — Cargar `frag_B` en nodo05.

Misma secuencia para el fragmento B.

Paso 10 — Verificar correctitud de fragmentación en ambos shards.

Confirmar completitud, disjunción, co-localización y que los JOINS locales producen resultados correctos sin acceder a datos de otro nodo.

Paso 11 — Demostrar la reconstrucción global desde nodo01.

Ejecutar consultas de conteo en los tres nodos, confirmar que frag_A + frag_B = total de nodo01, y mostrar la consulta UNION ALL que el coordinador ejecutará en la Fase 14.

Paso 12 — Limpieza en nodo01.

Eliminar tablas temporales y usuarios de transferencia.

Paso 13 — Apagar los nodos y tomar el snapshot `fase13-completa` en los cinco nodos.

D. Comandos completos

Los bloques **(host)** se ejecutan en PowerShell en Windows. Los bloques **(VM — nodoXX)** se ejecutan en una sesión SSH al nodo indicado. Los bloques SQL dentro de `sudo mariadb` se ejecutan en el prompt del motor.

D.1 Apagar todos los nodos activos (host)

 PowerShell

```
# Señal de apagado ordenado a cada nodo que pueda estar corriendo
# Los nodos que ya estén apagados devolverán un mensaje informativo sin error
VBoxManage controlvm "bdd-nodo01" acpipowerbutton 2>$null
VBoxManage controlvm "bdd-nodo02" acpipowerbutton 2>$null
VBoxManage controlvm "bdd-nodo03" acpipowerbutton 2>$null
# Esperar a que Ubuntu cierre limpiamente todos los servicios
Start-Sleep -Seconds 35
# Confirmar que no queda ninguna VM corriendo
VBoxManage list runningvms
```

La salida debe estar vacía. Si algún nodo persiste en la lista, esperar 15 segundos más y verificar de nuevo. En caso excepcional, forzar el estado guardado:

PowerShell

```
VBoxManage controlvm "bdd-nodo01" savestate
```

D.2 Crear `bdd-nodo04` como clon enlazado (host)

PowerShell

```
# Clon enlazado de nodo01 desde el snapshot fase08-completa.
# fase08-completa = Ubuntu Server + MariaDB 10.11 + lab_bdd poblado
#                               SIN ningún archivo de replicación (60-replication-*.cnf)
# → base ideal para un nodo shard autónomo
VBoxManage clonevm "bdd-nodo01" `
  --snapshot "fase08-completa" `
  --options linked `
  --name "bdd-nodo04" `
  --basefolder "C:\LabBDD\VMs" `
  --register
# Añadir descripción de rol para referencia futura
VBoxManage modifyvm "bdd-nodo04" `
  --description "SHARD-A - Fragmento horizontal frag_A (region norte+este). IP
192.168.56.104. Fase 13."
# Confirmar el registro
VBoxManage showvminfo "bdd-nodo04" | Select-String "Name:|State:|Memory:"
```

Salida esperada:

Text

```
Name:          bdd-nodo04
Memory size:   1536 MB
State:         powered off (...)
```

D.3 Configuración inicial de `bdd-nodo04` (host + VM)

Arrancar **solo nodo04** (nodo01 sigue apagado → no hay conflicto de IP):

PowerShell

```
VBoxManage startvm "bdd-nodo04" --type headless  
Start-Sleep -Seconds 35
```

Conectar por SSH a la IP heredada del clon (aún es .101, porque nodo01 está apagado):

PowerShell

```
ssh bddadmin@192.168.56.101
```

Ejecutar en la sesión SSH (**VM — nodo04**):

Bash

```
# — 1. Cambiar el hostname —————  
sudo hostnamectl set-hostname bdd-nodo04  
# El archivo /etc/hosts tiene "bdd-nodo01"; actualizarlo  
sudo sed -i 's/bdd-nodo01/bdd-nodo04/g' /etc/hosts  
# Verificar  
hostnamectl status | grep "Static hostname"  
# Esperado: Static hostname: bdd-nodo04
```

Bash

```
# — 2. Cambiar la IP estática de .101 a .104 —————  
# Identificar el archivo de configuración de netplan  
NETPLAN_FILE=$(ls /etc/netplan/*.yaml | head -1)  
echo "Archivo: $NETPLAN_FILE"  
# Ver configuración actual (mostrará 192.168.56.101)  
cat "$NETPLAN_FILE"  
# Reemplazar la dirección IP  
sudo sed -i 's/192\.\168\.\56\.\101/192.168.56.104/g' "$NETPLAN_FILE"  
# Confirmar el cambio antes de aplicar  
grep "192.168" "$NETPLAN_FILE"  
# Aplicar — la sesión SSH se interrumpirá en este momento  
sudo netplan apply
```

La sesión SSH **se pierde** al cambiar la IP. Reconectar desde el host a la nueva dirección:

 PowerShell

```
# Nueva ventana de PowerShell → conectar a la IP nueva de nodo04
ssh bddadmin@192.168.56.104
```

Continuar la configuración (**VM — nodo04**):

 Bash

```
# — 3. Verificar la nueva IP —————
ip addr show | grep "192.168.56"
# Esperado: inet 192.168.56.104/24
ping -c 2 192.168.56.1      # host Windows — debe responder
```

 Bash

```
# — 4. Regenerar las claves SSH del host del nodo —————
# El clon hereda las claves SSH de nodo01; se generan nuevas
# únicas para nodo04 (buena práctica de seguridad)
sudo rm -f /etc/ssh/ssh_host_*
sudo ssh-keygen -A
sudo systemctl restart ssh
echo "Claves SSH regeneradas:"
ls -la /etc/ssh/ssh_host_*.pub
```

 Bash

```
# — 5. Configurar MariaDB para el rol de shard —————
# a) Cambiar bind-address en 50-server.cnf para aceptar
#     conexiones remotas (heredado como 127.0.0.1 del clon)
sudo sed -i \
    's/^bind-address[] (:space:)*=.*\/bind-address = 0.0.0.0/' \
    /etc/mysql/mariadb.conf.d/50-server.cnf
# Verificar el cambio
grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf
# b) Crear archivo de configuración del shard
sudo tee /etc/mysql/mariadb.conf.d/61-shard-config.cnf > /dev/null << 'EOF'
# =====
# Configuración de nodo shard A — bdd-nodo04
# IP: 192.168.56.104 | server_id: 4
# Fase 13 — Fragmentación Horizontal (frag_A: norte+este)
# Mapa de server_id del laboratorio:
#     nodo01=1, nodo02=2, nodo03=3, nodo04=4, nodo05=5, nodo06=6
# =====
[mariadb]
server_id          = 4
# Binary log no requerido en shards para esta fase
# (se habilitará si se necesita replicación del shard en fases avanzadas)
skip_name_resolve = ON
EOF
# c) Reiniciar y verificar
sudo systemctl restart mariadb
sudo systemctl status mariadb --no-pager -l | head -5
sudo mariadb -e "SHOW VARIABLES LIKE 'server_id';"
sudo mariadb -e "SHOW VARIABLES LIKE 'bind_address';"
sudo mariadb -e "SHOW VARIABLES LIKE 'log_bin';"
```

Salida esperada:

- `server_id`: 4
- `bind_address`: 0.0.0.0
- `log_bin`: OFF (sin binary log en el shard)

 Bash

```
# — 6. Crear el usuario de verificación para nodo01 —————
# nodo01 usará este usuario en D.11 para leer datos del shard
# y verificar la reconstrucción UNION ALL
sudo mariadb << 'EOF'
CREATE USER IF NOT EXISTS 'shard_verify'@'192.168.56.101'
  IDENTIFIED BY 'ShardVerify_2025!';
-- El GRANT sobre lab_bdd se otorgará después de crear el esquema (D.7)
SELECT User, Host FROM mysql.user WHERE User = 'shard_verify';
EOF
```

Dejar la sesión SSH de nodo04 abierta. Abrir una **nueva ventana de PowerShell** para el siguiente paso.

D.4 Crear `bdd-nodo05` como clon enlazado (host — nueva ventana)

 PowerShell

```
# nodo04 está corriendo en .104; nodo01 sigue apagado
# nodo05 arrancará temporalmente con .101 (sin conflicto: nodo01 está off)
VBoxManage clonevm "bdd-nodo01" `
  --snapshot "fase08-completa" `
  --options linked `
  --name "bdd-nodo05" `
  --basefolder "C:\LabBDD\VMs" `
  --register
VBoxManage modifyvm "bdd-nodo05" `
  --description "SHARD-B – Fragmento horizontal frag_B (region sur+oeste). IP
192.168.56.105. Fase 13."
VBoxManage showvminfo "bdd-nodo05" | Select-String "Name:|State:"
```

D.5 Configuración inicial de `bdd-nodo05` (host + VM)

 PowerShell

```
VBoxManage startvm "bdd-nodo05" --type headless
Start-Sleep -Seconds 35
# nodo04 corre en .104; nodo01 sigue apagado
# → nodo05 arrancó con .101 sin conflicto
ssh bddadmin@192.168.56.101
```

(VM — nodo05):

 Bash

```
# — 1. Hostname —————
sudo hostnamectl set-hostname bdd-nodo05
sudo sed -i 's/bdd-nodo01/bdd-nodo05/g' /etc/hosts
hostnamectl status | grep "Static hostname"
```

 Bash

```
# — 2. Cambiar IP a .105 —————
NETPLAN_FILE=$(ls /etc/netplan/*.yaml | head -1)
sudo sed -i 's/192\.\168\.\56\.\101/192.168.56.105/g' "$NETPLAN_FILE"
grep "192.168" "$NETPLAN_FILE"
sudo netplan apply # la sesión SSH se interrumpe aquí
```

Reconectar a la nueva IP:

 PowerShell

```
ssh bddadmin@192.168.56.105
```

(VM — nodo05) — continuar:

 Bash

```
# — 3. Verificar IP —————  
ip addr show | grep "192.168.56"  
# Esperado: inet 192.168.56.105/24
```

 Bash

```
# — 4. Regenerar claves SSH —————  
sudo rm -f /etc/ssh/ssh_host_*  
sudo ssh-keygen -A  
sudo systemctl restart ssh
```

 Bash

```
# — 5. Configurar MariaDB —————  
sudo sed -i \  
    's/^bind-address[:space:]*=.*\/bind-address = 0.0.0.0/' \  
    /etc/mysql/mariadb.conf.d/50-server.cnf  
sudo tee /etc/mysql/mariadb.conf.d/61-shard-config.cnf > /dev/null << 'EOF'  
# =====  
# Configuración de nodo shard B — bdd-nodo05  
# IP: 192.168.56.105 | server_id: 5  
# Fase 13 — Fragmentación Horizontal (frag_B: sur+oeste)  
# =====  
[mariadb]  
server_id          = 5  
skip_name_resolve = ON  
EOF  
sudo systemctl restart mariadb  
sudo mariadb -e "SHOW VARIABLES LIKE 'server_id';"  
sudo mariadb -e "SHOW VARIABLES LIKE 'bind_address';"
```

 Bash

```
# — 6. Usuario de verificación —————
sudo mariadb << 'EOF'
CREATE USER IF NOT EXISTS 'shard_verify'@'192.168.56.101'
    IDENTIFIED BY 'ShardVerify_2025!';
SELECT User, Host FROM mysql.user WHERE User = 'shard_verify';
EOF
```

En este punto el laboratorio tiene tres nodos corriendo sin conflictos de IP:

Nodo	IP	Estado
bdd-nodo04	192.168.56.104	En ejecución, shard A configurado
bdd-nodo05	192.168.56.105	En ejecución, shard B configurado
bdd-nodo01	—	Apagado (se inicia en el siguiente paso)

D.6 Reiniciar nodo01 y preparar la distribución de datos (host + VM)

 PowerShell

```
VBoxManage startvm "bdd-nodo01" --type headless
Start-Sleep -Seconds 35
# Ahora los tres nodos corren en IPs distintas sin conflicto
ssh bddadmin@192.168.56.101
```

(VM — nodo01):

 Bash

```
# — 1. Verificar conectividad con los shards —————
ping -c 3 192.168.56.104    # nodo04
ping -c 3 192.168.56.105    # nodo05
# — 2. Confirmar que la replicación física sigue activa —————
sudo mariadb -e "SHOW MASTER STATUS\G"
# Debe mostrar File y Position activos
```

 Bash

```
# — 3. Crear el usuario que los shards usarán para leer datos
#       de nodo01 (estrategia pull: el shard conecta a nodo01
#       y tira los datos via mysqldump)
sudo mariadb << 'EOF'
CREATE USER IF NOT EXISTS 'shard_pull'@'192.168.56.104'
    IDENTIFIED BY 'ShardPull_2025!';
GRANT SELECT ON lab_bdd.* TO 'shard_pull'@'192.168.56.104';
CREATE USER IF NOT EXISTS 'shard_pull'@'192.168.56.105'
    IDENTIFIED BY 'ShardPull_2025!';
GRANT SELECT ON lab_bdd.* TO 'shard_pull'@'192.168.56.105';
FLUSH PRIVILEGES;
-- Verificar
SELECT User, Host FROM mysql.user WHERE User = 'shard_pull';
EOF
```

 Bash

```
# — 4. Materializar las tablas temporales de detalle_pedidos —
# detalle_pedidos no tiene columna region (fragmentación derivada).
# Se crean dos tablas temporales en nodo01 que materializan cada
# fragmento mediante un JOIN con pedidos.
# Estas tablas se exportan con mysqldump --no-create-info para
# que los shards puedan importarla con un simple sed del nombre.
#
# NOTA: estos CREATE TABLE se replicarán a nodo02/nodo03.
# Los correspondientes DROP TABLE al final de la fase también
# se replicarán, dejando ambos esclavos en estado limpio.
sudo mariadb lab_bdd << 'EOF'
DROP TABLE IF EXISTS tmp_detalle_frag_A;
CREATE TABLE tmp_detalle_frag_A
    ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    SELECT dp.id, dp.pedido_id, dp.producto_id,
           dp.cantidad, dp.precio_unitario, dp.subtotal
    FROM   detalle_pedidos dp
    JOIN   pedidos p ON p.id = dp.pedido_id
    WHERE  p.region IN ('norte', 'este');
DROP TABLE IF EXISTS tmp_detalle_frag_B;
CREATE TABLE tmp_detalle_frag_B
    ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
    SELECT dp.id, dp.pedido_id, dp.producto_id,
           dp.cantidad, dp.precio_unitario, dp.subtotal
    FROM   detalle_pedidos dp
    JOIN   pedidos p ON p.id = dp.pedido_id
    WHERE  p.region IN ('sur', 'oeste');
-- Verificar la distribución y que la suma es completa
SELECT 'tmp_detalle_frag_A' AS fragmento, COUNT(*) AS filas
FROM   tmp_detalle_frag_A
UNION ALL
SELECT 'tmp_detalle_frag_B', COUNT(*)
FROM   tmp_detalle_frag_B
UNION ALL
SELECT 'detalle_pedidos (total original)', COUNT(*)
FROM   detalle_pedidos;
EOF
```

Resultado esperado (los valores exactos dependen de los datos de la Fase 8):

- `tmp_detalle_frag_A`: ~18 filas

- `tmp_detalle_frag_B` : ~18 filas
 - `detalle_pedidos (total)` : 36 filas — la suma de A y B debe ser exactamente 36
-

D.7 Crear el esquema `lab_bdd` en nodo04 y nodo05

El DDL es **idéntico** en ambos shards. Ejecutar primero en nodo04 y luego en nodo05.

| *En nodo04 (VM — nodo04):*

 Bash

```
sudo mariadb << 'EOF'
-- Eliminar la base de datos completa heredada del clon
-- (contiene los 20 clientes, 10 productos, etc. de la Fase 8
-- pero SIN la columna ultima_modificacion añadida en la Fase 10)
DROP DATABASE IF EXISTS lab_bdd;
-- Crear el esquema del shard
CREATE DATABASE lab_bdd
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE lab_bdd;
-- =====
-- TABLA: clientes
-- SIN FK. INCLUYE ultima_modificacion (añadida en nodo01 en
-- la Fase 10 vía ALTER TABLE replicado).
-- =====
CREATE TABLE clientes (
    id                INT                NOT NULL AUTO_INCREMENT,
    nombre            VARCHAR(100)      NOT NULL,
    apellido          VARCHAR(100)      NOT NULL,
    email             VARCHAR(150),
    telefono          VARCHAR(20),
    region            ENUM('norte','sur','este','oeste') NOT NULL,
    ciudad            VARCHAR(100),
    fecha_alta        DATETIME DEFAULT CURRENT_TIMESTAMP,
    ultima_modificacion DATETIME DEFAULT CURRENT_TIMESTAMP
                                ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uk_email (email),
    KEY idx_region (region)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Fragmento horizontal de clientes - sin FK
(integridad distribuida)';
-- =====
-- TABLA: productos
-- Catálogo COMPLETO en ambos shards (temporal hasta Fase 14).
-- En la Fase 14 se reemplazará por los dos fragmentos verticales
-- (V_productos_basico en nodo04 / V_productos_detalle en nodo05).
-- =====
CREATE TABLE productos (
    id                INT                NOT NULL AUTO_INCREMENT,
    sku               VARCHAR(50)        NOT NULL,
    nombre            VARCHAR(200)       NOT NULL,
    categoria         VARCHAR(100),
```

```

    descripcion      TEXT,
    ficha_tecnica     TEXT,
    imagen_url        VARCHAR(500),
    precio            DECIMAL(10,2) NOT NULL,
    stock            INT          DEFAULT 0,
    peso_kg          DECIMAL(8,3),
    fecha_creacion    DATETIME     DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uk_sku (sku)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Catálogo completo – fragmentación vertical pendiente (Fase 14)';
-- =====
-- TABLA: pedidos
-- SIN FK hacia clientes.
-- La co-localización (mismo predicado region) garantiza que el
-- cliente referenciado por cada pedido existe en este mismo nodo.
-- =====
CREATE TABLE pedidos (
    id                INT          NOT NULL AUTO_INCREMENT,
    cliente_id        INT          NOT NULL,
    region            ENUM('norte','sur','este','oeste') NOT NULL,
    fecha_pedido      DATETIME     DEFAULT CURRENT_TIMESTAMP,
    estado            ENUM('pendiente','procesado','enviado',
                          'entregado','cancelado') DEFAULT 'pendiente',
    total             DECIMAL(10,2),
    PRIMARY KEY (id),
    KEY idx_cliente (cliente_id),
    KEY idx_region   (region)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Fragmento horizontal de pedidos – sin FK (integridad por
co-localización)';
-- =====
-- TABLA: detalle_pedidos
-- SIN FK hacia pedidos ni hacia productos.
-- Co-localizada con el fragmento de pedidos de este shard.
-- =====
CREATE TABLE detalle_pedidos (
    id                INT          NOT NULL AUTO_INCREMENT,
    pedido_id         INT          NOT NULL,
    producto_id       INT          NOT NULL,
    cantidad          INT          NOT NULL DEFAULT 1,
    precio_unitario   DECIMAL(10,2) NOT NULL,
    subtotal          DECIMAL(10,2),
    PRIMARY KEY (id),
    KEY idx_pedido    (pedido_id),

```

```
        KEY idx_producto (producto_id)
    ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
      COMMENT='Fragmento derivado de detalle_pedidos - co-localizado
con pedidos';
-- Verificar estructura resultante
SHOW TABLES IN lab_bdd;
EOF
```

En nodo05 (VM — nodo05): ejecutar el bloque anterior sin modificar ninguna línea — el esquema es idéntico en ambos shards. Solo el contenido de los datos diferirá tras la carga.

 Bash

```
# Este bloque se ejecuta en nodo05; es IDÉNTICO al de nodo04
sudo mariadb << 'EOF'
DROP DATABASE IF EXISTS lab_bdd;
CREATE DATABASE lab_bdd
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE lab_bdd;
CREATE TABLE clientes (
    id                INT                NOT NULL AUTO_INCREMENT,
    nombre            VARCHAR(100)      NOT NULL,
    apellido          VARCHAR(100)      NOT NULL,
    email             VARCHAR(150),
    telefono          VARCHAR(20),
    region            ENUM('norte', 'sur', 'este', 'oeste') NOT NULL,
    ciudad            VARCHAR(100),
    fecha_alta        DATETIME DEFAULT CURRENT_TIMESTAMP,
    ultima_modificacion DATETIME DEFAULT CURRENT_TIMESTAMP
                                ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uk_email (email),
    KEY idx_region (region)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Fragmento horizontal de clientes - sin FK
(integridad distribuida)';
CREATE TABLE productos (
    id                INT                NOT NULL AUTO_INCREMENT,
    sku               VARCHAR(50)        NOT NULL,
    nombre            VARCHAR(200)       NOT NULL,
    categoria         VARCHAR(100),
    descripcion       TEXT,
    ficha_tecnica     TEXT,
    imagen_url        VARCHAR(500),
    precio            DECIMAL(10,2)     NOT NULL,
    stock             INT                DEFAULT 0,
    peso_kg           DECIMAL(8,3),
    fecha_creacion    DATETIME           DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uk_sku (sku)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
  COMMENT='Catálogo completo - fragmentación vertical pendiente (Fase 14)';
CREATE TABLE pedidos (
    id                INT                NOT NULL AUTO_INCREMENT,
    cliente_id        INT                NOT NULL,
```

```

region          ENUM('norte','sur','este','oeste') NOT NULL,
fecha_pedido    DATETIME          DEFAULT CURRENT_TIMESTAMP,
estado          ENUM('pendiente','procesado','enviado',
                    'entregado','cancelado') DEFAULT 'pendiente',

total          DECIMAL(10,2),
PRIMARY KEY (id),
KEY idx_cliente (cliente_id),
KEY idx_region  (region)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='Fragmento horizontal de pedidos – sin FK (integridad por
co-localización)';
CREATE TABLE detalle_pedidos (
    id            INT            NOT NULL AUTO_INCREMENT,
    pedido_id     INT            NOT NULL,
    producto_id   INT            NOT NULL,
    cantidad      INT            NOT NULL DEFAULT 1,
    precio_unitario DECIMAL(10,2) NOT NULL,
    subtotal      DECIMAL(10,2),
    PRIMARY KEY (id),
    KEY idx_pedido (pedido_id),
    KEY idx_producto (producto_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT='Fragmento derivado de detalle_pedidos – co-localizado
con pedidos';
SHOW TABLES IN lab_bdd;
EOF

```

Otorgar el privilegio SELECT al usuario `shard_verify` (creado en D.3/D.5) ahora que el esquema existe. Ejecutar **en nodo04 y en nodo05**:

 Bash

```
# En nodo04:
sudo mariadb << 'EOF'
GRANT SELECT ON lab_bdd.* TO 'shard_verify'@'192.168.56.101';
FLUSH PRIVILEGES;
SHOW GRANTS FOR 'shard_verify'@'192.168.56.101';
EOF

# En nodo05 (sesión SSH separada):
sudo mariadb << 'EOF'
GRANT SELECT ON lab_bdd.* TO 'shard_verify'@'192.168.56.101';
FLUSH PRIVILEGES;
SHOW GRANTS FOR 'shard_verify'@'192.168.56.101';
EOF
```

D.8 Cargar `frag_A` en nodo04 (VM — nodo04)

Los cuatro bloques siguientes se ejecutan desde la sesión SSH de **nodo04**.

`mysqldump` conecta hacia nodo01 (fuente) y la salida se canaliza directamente al cliente local `sudo mariadb lab_bdd`.

 Bash

```
# — 1. Clientes — fragmento A —————
# --no-create-info : solo INSERT INTO, no CREATE TABLE
# --skip-triggers  : no copiar triggers del esquema centralizado
# --where          : predicado del fragmento A según el DDD
mysqldump \
  -h 192.168.56.101 \
  -u shard_pull \
  -p'ShardPull_2025!' \
  --no-create-info \
  --skip-triggers \
  --where="region IN ('norte','este')" \
  lab_bdd clientes | \
  sudo mariadb lab_bdd
echo ">>> Clientes frag_A importados."
sudo mariadb lab_bdd -e "SELECT COUNT(*) AS clientes_cargados FROM clientes;"
```

 Bash

```
# — 2. Pedidos – fragmento A —————
mysqldump \
  -h 192.168.56.101 \
  -u shard_pull \
  -p'ShardPull_2025!' \
  --no-create-info \
  --skip-triggers \
  --where="region IN ('norte','este')" \
  lab_bdd pedidos | \
  sudo mariadb lab_bdd
echo ">>> Pedidos frag_A importados."
sudo mariadb lab_bdd -e "SELECT COUNT(*) AS pedidos_cargados FROM pedidos;"
```

 Bash

```
# — 3. detalle_pedidos – fragmento derivado A —————
# Se exporta la tabla temporal tmp_detalle_frag_A (creada en D.6)
# y sed renombra la tabla en los INSERT antes de importar.
# Resultado: INSERT INTO `detalle_pedidos` (...) en lugar de
#             INSERT INTO `tmp_detalle_frag_A` (...)
mysqldump \
  -h 192.168.56.101 \
  -u shard_pull \
  -p'ShardPull_2025!' \
  --no-create-info \
  --skip-triggers \
  lab_bdd tmp_detalle_frag_A | \
  sed 's/`tmp_detalle_frag_A`/`detalle_pedidos`/g' | \
  sudo mariadb lab_bdd
echo ">>> detalle_pedidos frag_A importado."
sudo mariadb lab_bdd -e "SELECT COUNT(*) AS detalles_cargados
FROM detalle_pedidos;"
```

 Bash

```
# — 4. Productos – catálogo COMPLETO (10 filas) —————
# Sin filtro --where: se cargan todos los productos.
# Esto es temporal; en la Fase 14 productos se fragmentará
# verticalmente y este bloque se eliminará.
mysqldump \
  -h 192.168.56.101 \
  -u shard_pull \
  -p'ShardPull_2025!' \
  --no-create-info \
  --skip-triggers \
  lab_bdd productos | \
  sudo mariadb lab_bdd
echo ">>> Productos importados."
sudo mariadb lab_bdd -e "SELECT COUNT(*) AS productos_cargados
FROM productos;"
```

 Bash

```
# — 5. Resumen de carga en nodo04 —————
sudo mariadb lab_bdd -e "
SELECT 'clientes'          AS tabla, COUNT(*) AS filas FROM clientes
UNION ALL
SELECT 'productos',        COUNT(*)          FROM productos
UNION ALL
SELECT 'pedidos',          COUNT(*)          FROM pedidos
UNION ALL
SELECT 'detalle_pedidos',  COUNT(*)          FROM detalle_pedidos;"
```

Resultado esperado en nodo04:

Text

```
+-----+-----+
| tabla          | filas |
+-----+-----+
| clientes       | 10    | ← frag_A: norte+este únicamente
| productos     | 10    | ← catálogo completo (temporal)
| pedidos       | 10    | ← frag_A: norte+este únicamente
| detalle_pedidos | 18    | ← aprox.; co-localizado con pedidos_frag_A
+-----+-----+
```

(el número exacto de detalle_pedidos puede ser 17, 18 o 19 según la distribución aleatoria de los datos de prueba generados en la Fase 8)

D.9 Cargar frag_B en nodo05 (VM — nodo05)

Bash

```
# — 1. Clientes - fragmento B —————
mysqldump \
  -h 192.168.56.101 \
  -u shard_pull \
  -p'ShardPull_2025!' \
  --no-create-info \
  --skip-triggers \
  --where="region IN ('sur','oeste')" \
  lab_bdd clientes | \
  sudo mariadb lab_bdd
echo ">>> Clientes frag_B importados."
sudo mariadb lab_bdd -e "SELECT COUNT(*) AS clientes_cargados FROM clientes;"
```

 Bash

```
# — 2. Pedidos - fragmento B —————
mysqldump \
  -h 192.168.56.101 \
  -u shard_pull \
  -p'ShardPull_2025!' \
  --no-create-info \
  --skip-triggers \
  --where="region IN ('sur','oeste')" \
  lab_bdd pedidos | \
  sudo mariadb lab_bdd
echo ">>> Pedidos frag_B importados."
sudo mariadb lab_bdd -e "SELECT COUNT(*) AS pedidos_cargados FROM pedidos;"
```

 Bash

```
# — 3. detalle_pedidos - fragmento derivado B —————
mysqldump \
  -h 192.168.56.101 \
  -u shard_pull \
  -p'ShardPull_2025!' \
  --no-create-info \
  --skip-triggers \
  lab_bdd tmp_detalle_frag_B | \
  sed 's/`tmp_detalle_frag_B`/`detalle_pedidos`/g' | \
  sudo mariadb lab_bdd
echo ">>> detalle_pedidos frag_B importado."
sudo mariadb lab_bdd -e "SELECT COUNT(*) AS detalles_cargados
FROM detalle_pedidos;"
```

 Bash

```
# — 4. Productos – catálogo COMPLETO —————
mysqldump \
  -h 192.168.56.101 \
  -u shard_pull \
  -p'ShardPull_2025!' \
  --no-create-info \
  --skip-triggers \
  lab_bdd productos | \
  sudo mariadb lab_bdd
echo ">>> Productos importados."
```

 Bash

```
# — 5. Resumen de carga en nodo05 —————
sudo mariadb lab_bdd -e "
SELECT 'clientes'          AS tabla, COUNT(*) AS filas FROM clientes
UNION ALL
SELECT 'productos',        COUNT(*)          FROM productos
UNION ALL
SELECT 'pedidos',          COUNT(*)          FROM pedidos
UNION ALL
SELECT 'detalle_pedidos',  COUNT(*)          FROM detalle_pedidos;"
```

Resultado esperado en nodo05 (espejo de nodo04 pero con datos de frag_B):

 Text

```
+-----+-----+
| tabla          | filas |
+-----+-----+
| clientes        | 10    | ← frag_B: sur+oeste únicamente
| productos       | 10    | ← catálogo completo (temporal)
| pedidos         | 10    | ← frag_B: sur+oeste únicamente
| detalle_pedidos | 18    | ← aprox.; co-localizado con pedidos_frag_B
+-----+-----+
```

D.10 Verificar correctitud de fragmentación en los shards

Ejecutar el siguiente bloque **en nodo04**. Para nodo05, cambiar los valores del CASE de `'norte', 'este'` por `'sur', 'oeste'` (y el mensaje de texto, si se desea).

(VM — nodo04):

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- VERIFICACIÓN 1: DISJUNCIÓN
-- Las regiones presentes deben ser SOLO las del fragmento asignado.
-- En nodo04 deben aparecer únicamente 'norte' y 'este'.
-- =====
SELECT 'Disjunción - clientes'          AS verificacion,
       GROUP_CONCAT(DISTINCT region
                    ORDER BY region)    AS regiones_presentes,
CASE
  WHEN GROUP_CONCAT(DISTINCT region ORDER BY region)
       IN ('este,norte', 'norte,este')
  THEN 'OK - solo frag_A (norte+este)'
  ELSE 'FALLA - hay regiones que no pertenecen a este shard'
END AS resultado
FROM clientes;
-- Buscar explícitamente filas de frag_B que no deberían estar aquí
SELECT 'Filas de frag_B en nodo04'      AS verificacion,
       COUNT(*)                          AS filas_invalidas,
CASE WHEN COUNT(*) = 0 THEN 'OK'
      ELSE 'FALLA' END                  AS resultado
FROM clientes
WHERE region IN ('sur', 'oeste');
-- Lo mismo para pedidos
SELECT 'Disjunción - pedidos'           AS verificacion,
       GROUP_CONCAT(DISTINCT region
                    ORDER BY region)    AS regiones_presentes,
CASE
  WHEN GROUP_CONCAT(DISTINCT region ORDER BY region)
       IN ('este,norte', 'norte,este')
  THEN 'OK - solo frag_A'
  ELSE 'FALLA'
END AS resultado
FROM pedidos;
-- =====
-- VERIFICACIÓN 2: CO-LOCALIZACIÓN detalle_pedidos ↔ pedidos
-- Todos los pedidos referenciados en detalle_pedidos deben
-- tener su fila padre en la tabla pedidos de este mismo nodo.
-- =====
SELECT 'Co-localización detalle↔pedidos' AS verificacion,
       COUNT(*)                          AS detalles_sin_pedido_local,
CASE
  WHEN COUNT(*) = 0
```

```

        THEN 'OK - co-localización correcta'
        ELSE 'FALLA - línea de detalle huérfana detectada'
    END AS resultado
FROM detalle_pedidos dp
WHERE NOT EXISTS (
    SELECT 1 FROM pedidos p WHERE p.id = dp.pedido_id
);
-- =====
-- VERIFICACIÓN 3: INTEGRIDAD CLIENTE-PEDIDO
-- Verificar que la co-localización preserva la integridad:
-- el cliente de cada pedido debe existir en este mismo shard.
-- (No hay FK de motor, pero el diseño lo garantiza por predicado.)
-- =====
SELECT 'Integridad cliente-pedido' AS verificacion,
       COUNT(*) AS pedidos_sin_cliente_local,
       CASE
           WHEN COUNT(*) = 0
           THEN 'OK - integridad preservada por co-localización'
           ELSE 'ADVERTENCIA - revisar predicado de fragmentación'
       END AS resultado
FROM pedidos p
WHERE NOT EXISTS (
    SELECT 1 FROM clientes c WHERE c.id = p.cliente_id
);
-- =====
-- VERIFICACIÓN 4: JOIN LOCAL COMPLETO
-- El JOIN de las tres tablas debe resolverse sin datos externos.
-- Resultado: solo filas de regiones norte y este.
-- =====
SELECT c.region,
       CONCAT(c.nombre, ' ', c.apellido) AS cliente,
       p.id AS pedido_id,
       p.estado,
       COUNT(dp.id) AS lineas_detalle,
       ROUND(SUM(dp.subtotal), 2) AS total_calculado,
       p.total AS total_registrado
FROM clientes c
JOIN pedidos p ON p.cliente_id = c.id
JOIN detalle_pedidos dp ON dp.pedido_id = p.id
GROUP BY c.region, c.id, p.id
ORDER BY c.region, p.id;
EOF

```

En nodo05, ejecutar el mismo bloque ajustando los textos del CASE:
cambiar `IN ('sur','oeste')` como valores válidos y `IN ('norte','este')`
como inválidos. El resultado esperado debe mostrar solo regiones `sur` y `oeste`.

D.11 Verificar la reconstrucción global desde nodo01

(VM — nodo01):

 Bash

```
sudo mariadb lab_bdd << 'EOF'
-- =====
-- VERIFICACIÓN DE COMPLETITUD – TRES CONDICIONES FORMALES
-- Se simula en nodo01 la reconstrucción que en la Fase 14
-- realizará el coordinador (nodo06) con el motor Spider.
-- =====
-- --- CLIENTES ---
SELECT 'Completitud clientes' AS condicion,
      (SELECT COUNT(*) FROM clientes) AS total_global,
      (SELECT COUNT(*) FROM clientes WHERE region IN ('norte','este')) +
      (SELECT COUNT(*) FROM clientes WHERE region IN ('sur','oeste'))
AS suma_fragmentos,
CASE
  WHEN (SELECT COUNT(*) FROM clientes) =
        (SELECT COUNT(*) FROM clientes WHERE region IN
('norte','este')) +
        (SELECT COUNT(*) FROM clientes WHERE region IN ('sur','oeste'))
  THEN 'OK'
  ELSE 'FALLA'
END AS resultado
UNION ALL
SELECT 'Disjunción clientes',
      0,
      (SELECT COUNT(*) FROM clientes
      WHERE region IN ('norte','este') AND region IN ('sur','oeste')),
CASE
  WHEN (SELECT COUNT(*) FROM clientes
        WHERE region IN ('norte','este')
        AND region IN ('sur','oeste')) = 0
  THEN 'OK'
  ELSE 'FALLA'
END
UNION ALL
-- --- PEDIDOS ---
SELECT 'Completitud pedidos',
      (SELECT COUNT(*) FROM pedidos),
      (SELECT COUNT(*) FROM pedidos WHERE region IN ('norte','este')) +
      (SELECT COUNT(*) FROM pedidos WHERE region IN ('sur','oeste')),
CASE
  WHEN (SELECT COUNT(*) FROM pedidos) =
        (SELECT COUNT(*) FROM pedidos WHERE region IN
('norte','este')) +
        (SELECT COUNT(*) FROM pedidos WHERE region IN ('sur','oeste'))
```

```

        THEN 'OK' ELSE 'FALLA'
    END
UNION ALL
-- --- DETALLE_PEDIDOS (fragmentación derivada) ---
SELECT 'Compleitud detalle_pedidos',
    (SELECT COUNT(*) FROM detalle_pedidos),
    (SELECT COUNT(*) FROM tmp_detalle_frag_A) +
    (SELECT COUNT(*) FROM tmp_detalle_frag_B),
CASE
    WHEN (SELECT COUNT(*) FROM detalle_pedidos) =
        (SELECT COUNT(*) FROM tmp_detalle_frag_A) +
        (SELECT COUNT(*) FROM tmp_detalle_frag_B)
    THEN 'OK' ELSE 'FALLA'
END;
EOF

```

 Bash

```
# — Consulta UNION ALL de reconstrucción (anticipación de la Fase 14) —
# Esta es la consulta exacta que el coordinador ejecutará via Spider.
# Por ahora se valida en nodo01 sobre sus propios datos.
sudo mariadb lab_bdd << 'EOF'
-- Reconstrucción de clientes: UNION ALL de frag_A + frag_B
SELECT COUNT(*) AS clientes_reconstruidos
FROM (
    SELECT id, nombre, apellido, region FROM clientes
    WHERE region IN ('norte','este')      -- subconsulta → nodo04 en Fase 14
    UNION ALL
    SELECT id, nombre, apellido, region FROM clientes
    WHERE region IN ('sur','oeste')      -- subconsulta → nodo05 en Fase 14
) AS t_reconstruido;
-- Debe devolver 20: igual al total de la tabla original
-- Resumen de distribución por región
SELECT region,
    COUNT(*) AS clientes,
    CASE
        WHEN region IN ('norte','este') THEN 'nodo04 (frag_A)'
        ELSE 'nodo05 (frag_B)'
    END AS shard_destino
FROM clientes
GROUP BY region
ORDER BY region;
EOF
```

 Bash

```
# — Verificación cruzada de conteos consultando los shards remotamente —
# nodo01 conecta a nodo04 y nodo05 usando shard_verify
echo "=== nodo04 (frag_A) ==="
mysql -h 192.168.56.104 \
    -u shard_verify \
    -p'ShardVerify_2025!' \
    lab_bdd \
    -e "SELECT 'clientes'          AS t, COUNT(*) AS n FROM clientes
UNION ALL
      SELECT 'pedidos',          COUNT(*)   FROM pedidos
UNION ALL
      SELECT 'detalle_pedidos',  COUNT(*)   FROM detalle_pedidos
UNION ALL
      SELECT 'productos',        COUNT(*)   FROM
productos;" 2>/dev/null
echo ""
echo "=== nodo05 (frag_B) ==="
mysql -h 192.168.56.105 \
    -u shard_verify \
    -p'ShardVerify_2025!' \
    lab_bdd \
    -e "SELECT 'clientes'          AS t, COUNT(*) AS n FROM clientes
UNION ALL
      SELECT 'pedidos',          COUNT(*)   FROM pedidos
UNION ALL
      SELECT 'detalle_pedidos',  COUNT(*)   FROM detalle_pedidos
UNION ALL
      SELECT 'productos',        COUNT(*)   FROM
productos;" 2>/dev/null
echo ""
echo "=== nodo01 (global - referencia) ==="
sudo mariadb lab_bdd \
    -e "SELECT 'clientes'          AS t, COUNT(*) AS n FROM clientes
UNION ALL
      SELECT 'pedidos',          COUNT(*)   FROM pedidos
UNION ALL
      SELECT 'detalle_pedidos',  COUNT(*)   FROM detalle_pedidos
UNION ALL
      SELECT 'productos',        COUNT(*)   FROM productos;"
```

Resultado esperado (los detalles pueden ser 17–19 según los datos de la Fase 8):

Text

```
nodo04  clientes=10  pedidos=10  detalle~18  productos=10
nodo05  clientes=10  pedidos=10  detalle~18  productos=10
nodo01  clientes=20  pedidos=20  detalle=36  productos=10
nodo04 + nodo05 = nodo01 → Completitud verificada ✓
```

D.12 Limpieza en nodo01 (VM — nodo01)

Bash

```
sudo mariadb << 'EOF'
-- Eliminar tablas temporales (los datos ya están cargados en los shards)
DROP TABLE IF EXISTS lab_bdd.tmp_detalle_frag_A;
DROP TABLE IF EXISTS lab_bdd.tmp_detalle_frag_B;
-- Eliminar usuarios de transferencia (principio de mínimo privilegio:
-- una vez completada la carga, el acceso ya no es necesario)
DROP USER IF EXISTS 'shard_pull'@'192.168.56.104';
DROP USER IF EXISTS 'shard_pull'@'192.168.56.105';
FLUSH PRIVILEGES;
-- Confirmar que lab_bdd conserva únicamente sus tablas originales
SHOW TABLES IN lab_bdd;
-- Confirmar que los datos originales siguen intactos
SELECT 'clientes'          AS tabla, COUNT(*) AS filas FROM lab_bdd.clientes
UNION ALL
SELECT 'productos',        COUNT(*)          FROM lab_bdd.productos
UNION ALL
SELECT 'pedidos',          COUNT(*)          FROM lab_bdd.pedidos
UNION ALL
SELECT 'detalle_pedidos',  COUNT(*)          FROM lab_bdd.detalle_pedidos;
EOF
```

Salida esperada:

- `SHOW TABLES` devuelve las cuatro tablas originales sin `tmp_detalle_frag_*`.
- Los conteos son 20 / 10 / 20 / 36, idénticos al estado post-Fase 8.

Los `DROP TABLE` y `DROP USER` de este paso se propagan automáticamente a `bdd-nodo02` y `bdd-nodo03` vía replicación física y lógica, respectivamente. Los esclavos eliminan las tablas temporales que recibieron cuando se crearon, quedando con `lab_bdd` en su estado limpio original. Comportamiento correcto y esperado.

D.13 Apagar los nodos y tomar los snapshots `fase13-completa` (host)

 Bash

```
# En la sesión SSH de nodo01:  
sudo poweroff
```

 Bash

```
# En la sesión SSH de nodo04:  
sudo poweroff
```

 Bash

```
# En la sesión SSH de nodo05:  
sudo poweroff
```

Confirmar desde el host que las VMs están detenidas:

 PowerShell

```
VMBoxManage list runningvms  
# Salida esperada: vacía
```

Tomar el snapshot en los cinco nodos del laboratorio:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" take "fase13-completa" `
--description "MAESTRO: lab_bdd intacto (20/10/20/36). tmp_detalle
eliminadas. shard_pull eliminado. Fragmentación horizontal distribuida a
nodo04/05 verificada."
VBoxManage snapshot "bdd-nodo02" take "fase13-completa" `
--description "ESCLAVO: lab_bdd igual que nodo01 post-limpieza. Sin cambios
funcionales en esta fase."
VBoxManage snapshot "bdd-nodo03" take "fase13-completa" `
--description "MULTI-MAESTRO: lab_bdd igual que nodo01 post-limpieza. Sin
cambios funcionales en esta fase."
VBoxManage snapshot "bdd-nodo04" take "fase13-completa" `
--description "SHARD-A: lab_bdd con frag_A (norte+este). clientes=10,
pedidos=10, detalle~18, productos=10. server_id=4. IP=192.168.56.104. Sin FK."
VBoxManage snapshot "bdd-nodo05" take "fase13-completa" `
--description "SHARD-B: lab_bdd con frag_B (sur+oeste). clientes=10,
pedidos=10, detalle~18, productos=10. server_id=5. IP=192.168.56.105. Sin FK."
```

Verificar la lista de snapshots:

 PowerShell

```
VBoxManage snapshot "bdd-nodo01" list
VBoxManage snapshot "bdd-nodo02" list
VBoxManage snapshot "bdd-nodo03" list
VBoxManage snapshot "bdd-nodo04" list
VBoxManage snapshot "bdd-nodo05" list
```

`bdd-nodo01` y `bdd-nodo02` deben mostrar siete snapshots (`fase05-completa` hasta `fase13-completa`). `bdd-nodo03` debe mostrar desde `fase11-completa` hasta `fase13-completa` . `bdd-nodo04` y `bdd-nodo05` muestran únicamente su snapshot inicial `fase13-completa` .

E. Verificación de funcionamiento

Esta fase se considera completa cuando se cumplen **todos** los puntos siguientes:

1. `VBoxManage showvminfo "bdd-nodo04"` y `"bdd-nodo05"` confirman que las VMs están registradas con las descripciones de rol de shard y en estado `powered off` .

2. `hostnamectl status` devuelve `bdd-nodo04` en nodo04 y `bdd-nodo05` en nodo05.
3. `ip addr show | grep 192.168` muestra `192.168.56.104/24` en nodo04 y `192.168.56.105/24` en nodo05.
4. `SHOW VARIABLES LIKE 'server_id'` devuelve `4` en nodo04 y `5` en nodo05.
5. `SHOW VARIABLES LIKE 'log_bin'` devuelve `OFF` en nodo04 y nodo05.
6. `SHOW VARIABLES LIKE 'bind_address'` devuelve `0.0.0.0` en nodo04 y nodo05.
7. `SHOW TABLES IN lab_bdd` en nodo04 y nodo05 devuelve exactamente cuatro tablas: `clientes`, `detalle_pedidos`, `pedidos`, `productos`.
8. `SHOW CREATE TABLE pedidos` y `SHOW CREATE TABLE detalle_pedidos` en ambos shards **no contienen** ninguna cláusula `FOREIGN KEY` ni `CONSTRAINT ... FOREIGN`.
9. `SELECT COUNT(*) FROM clientes` en nodo04 devuelve exactamente `10` filas.
10. `SELECT DISTINCT region FROM clientes` en nodo04 muestra **solo** `norte` y `este` — ningún valor de `frag_B`.
11. `SELECT COUNT(*) FROM clientes` en nodo05 devuelve exactamente `10` filas.
12. `SELECT DISTINCT region FROM clientes` en nodo05 muestra **solo** `sur` y `oeste` — ningún valor de `frag_A`.
13. `SELECT COUNT(*) FROM pedidos` devuelve `10` en nodo04 y `10` en nodo05.
14. La suma `COUNT(detalle_pedidos nodo04) + COUNT(detalle_pedidos nodo05)` es igual al total de `detalle_pedidos` en nodo01 (36 filas).
15. `SELECT COUNT(*) FROM productos` devuelve `10` en nodo04 y `10` en nodo05 (catálogo completo en ambos).
16. La verificación de disjunción: `SELECT COUNT(*) FROM clientes WHERE region IN ('sur','oeste')` en nodo04 devuelve `0`; lo mismo para `('norte','este')` en nodo05.
17. La verificación de co-localización: `SELECT COUNT(*) FROM detalle_pedidos dp WHERE NOT EXISTS (SELECT 1 FROM pedidos p WHERE p.id = dp.pedido_id)` devuelve `0` en ambos shards.
18. La verificación de integridad cliente-pedido: `SELECT COUNT(*) FROM pedidos p WHERE NOT EXISTS (SELECT 1 FROM clientes c WHERE c.id = p.cliente_id)` devuelve `0` en ambos shards.
19. El JOIN local completo (D.10) en nodo04 produce únicamente filas con regiones `norte` y `este`; en nodo05 únicamente `sur` y `oeste`.
20. Las verificaciones de completitud en nodo01 (D.11) devuelven `OK` para las tres condiciones y las tres tablas fragmentadas.
21. `SHOW TABLES IN lab_bdd` en nodo01 no muestra `tmp_detalle_frag_A` ni `tmp_detalle_frag_B`.

22. `SHOW GRANTS FOR 'shard_pull'@'192.168.56.104'` en nodo01 devuelve error o resultado vacío (usuario eliminado en D.12).
 23. Los snapshots `fase13-completa` existen en los cinco nodos del laboratorio.
 24. El estudiante puede explicar de memoria la diferencia entre fragmentación primaria (`clientes` , `pedidos`) y derivada (`detalle_pedidos`), y por qué `productos` se carga completo temporalmente en ambos shards.
-

F. Problemas comunes y soluciones

Problema	Causa probable	Solución
SSH a .101 para configurar nodo04 conecta al nodo equivocado o da <code>Connection refused</code>	nodo01 sigue corriendo con .101 al mismo tiempo que el clon, generando conflicto ARP	Apagar nodo01 completamente antes de arrancar nodo04 (D.1); verificar con <code>VBoxManage list runningvms</code> antes de D.3
<code>VBoxManage clonevm</code> falla con "Could not find a snapshot named 'fase08-completa'"	El nombre del snapshot no coincide exactamente (espacios, mayúsculas, acentos)	Ejecutar <code>VBoxManage snapshot "bdd-nodo01" list</code> para ver el nombre exacto; copiarlo literalmente en el comando
<code>netplan apply</code> produce error de sintaxis YAML	Edición manual incorrecta del archivo <code>.yaml</code> (indentación YAML es sensible)	Usar exactamente el comando <code>sed -i 's/192\.168\.56\.101/192.168.56.104/g'</code> en lugar de editar manualmente; validar con <code>sudo netplan try</code> antes de aplicar
La sesión SSH se pierde durante <code>netplan apply</code> y la nueva IP no responde	El cambio de IP se aplicó pero hay un error de red adicional (gateway, interfaz)	Usar la consola VirtualBox (botón "Show" en VirtualBox Manager) para iniciar sesión directamente; ejecutar <code>ip</code>

		<pre>addr show y cat /etc/netplan/* .yaml para diagnosticar</pre>
<pre>mysqldump produce ERROR 2003: Can't connect to MySQL server on '192.168.56.10 1'</pre>	<pre>bind-address sigue siendo 127.0.0.1 en nodo01 (no se cambió en Fase 10), o el usuario shard_pull no existe</pre>	Verificar en nodo01: <code>grep bind-address /etc/mysql/mariadb.conf.d/50-server.cnf</code> ; si sigue en 127.0.0.1, ejecutar el <code>sed</code> del D.2 de la Fase 10 y reiniciar MariaDB
<pre>mysqldump devuelve Access denied for user 'shard_pull'@' 192.168.56.104 '</pre>	El usuario fue creado con host incorrecto o sin el GRANT correspondiente	En nodo01: <pre>SHOW GRANTS FOR 'shard_pull'@' 192.168.56.104 '; si falta: GRANT SELECT ON lab_bdd.* TO 'shard_pull'@' 192.168.56.104 '; FLUSH PRIVILEGES;</pre>
El <code>sed</code> del paso de <code>tmp_detalle_fr ag_A</code> no reemplaza el nombre y los INSERT fallan con "Table doesn't exist"	Las comillas invertidas o el patrón del <code>sed</code> no coinciden con la salida real de <code>mysqldump</code>	Probar primero solo el <code>mysqldump head -30</code> para ver el formato exacto de los INSERT; ajustar el patrón del <code>sed</code> si el nombre de tabla aparece entre

		comillas simples en lugar de invertidas
<code>SHOW CREATE</code> <code>TABLE pedidos</code> en el shard muestra <code>FOREIGN KEY</code>	Se usó <code>mysqldump</code> para importar el esquema en lugar del DDL explícito de D.7	Ejecutar <code>DROP</code> <code>TABLE pedidos;</code> <code>DROP TABLE</code> <code>detalle_pedido</code> <code>s;</code> en el shard afectado, volver a crear con el DDL de D.7 (sin FK) y reimportar los datos
La verificación de co-localización devuelve filas > 0 en nodo04	Se cargó <code>tmp_detalle_fr</code> <code>ag_B</code> en nodo04 en lugar de <code>tmp_detalle_fr</code> <code>ag_A</code> (error en el sed o en la selección de la tabla)	<code>TRUNCATE TABLE</code> <code>detalle_pedido</code> <code>s;</code> en nodo04 y repetir el paso D.8 bloque 3 con la tabla correcta (<code>tmp_detalle_f</code> <code>rag_A</code>)
La suma de <code>detalle_pedido</code> <code>s</code> en nodo04 + nodo05 no suma 36	Las tablas temporales en nodo01 no materializaron correctamente todos los pedidos	En nodo01: <code>SELECT</code> <code>COUNT(*) FROM</code> <code>tmp_detalle_fr</code> <code>ag_A;</code> <code>SELECT</code> <code>COUNT(*) FROM</code> <code>tmp_detalle_fr</code> <code>ag_B;</code> — la suma debe ser 36; si no, ejecutar <code>DROP</code> <code>TABLE IF</code> <code>EXISTS</code> <code>tmp_detalle_fr</code> <code>ag_A;</code> <code>CREATE</code> <code>TABLE...</code> con el JOIN corregido (D.6 paso 4)

<code>VBoxManage snapshot ... take</code> falla con “Cannot take a snapshot of the machine while it is running”	La VM está encendida al momento de tomar el snapshot	Confirmar con <code>VBoxManage list runningvms</code> que está vacío; si la VM aparece, apagar con <code>VBoxManage controlvm "bdd-nodo04" acpipowerbutton</code> y esperar
nodo02 o nodo03 muestran error en el hilo SQL al reiniciar (por las tablas <code>tmp_detalle_frag_*</code>)	El esclavo recibió el <code>CREATE TABLE</code> pero aplicó el <code>DROP TABLE</code> antes de terminar de procesar eventos intermedios	En nodo02: <code>SHOW SLAVE STATUS\G</code>; si <code>Last_SQL_Error</code> menciona “Table doesn’t exist” en el DROP: <code>SET GLOBAL SQL_SLAVE_SKIP_COUNTER = 1; START SLAVE;</code> — el estado final (sin tablas tmp) es el correcto
<code>mysql -h 192.168.56.104 -u shard_verify</code> desde nodo01 da <code>Access denied</code>	El usuario <code>shard_verify</code> se creó en D.3 pero el GRANT sobre <code>lab_bdd.*</code> no se otorgó en D.7	En nodo04: <code>GRANT SELECT ON lab_bdd.* TO 'shard_verify'@'192.168.56.101'; FLUSH PRIVILEGES;</code>

G. Checklist de validación

- ☐ `VBoxManage showvminfo "bdd-nodo04"` y `"bdd-nodo05"` confirman las VMs como clones enlazados registrados con la descripción de rol de shard.
- ☐ `hostnamectl status` devuelve `bdd-nodo04` en nodo04 y `bdd-nodo05` en nodo05.

- ☐ `ip addr show` muestra `.104` en nodo04 y `.105` en nodo05.
- ☐ `SHOW VARIABLES LIKE 'server_id'` devuelve `4` en nodo04 y `5` en nodo05.
- ☐ `SHOW VARIABLES LIKE 'log_bin'` devuelve `OFF` en ambos shards.
- ☐ `SHOW VARIABLES LIKE 'bind_address'` devuelve `0.0.0.0` en ambos shards.
- ☐ `SHOW TABLES IN lab_bdd` en ambos shards devuelve exactamente cuatro tablas (`clientes`, `detalle_pedidos`, `pedidos`, `productos`).
- ☐ `SHOW CREATE TABLE pedidos` y `SHOW CREATE TABLE detalle_pedidos` en ambos shards **no contienen** `FOREIGN KEY`.
- ☐ `SELECT COUNT(*) FROM clientes` devuelve `10` en nodo04 y `10` en nodo05.
- ☐ `SELECT DISTINCT region FROM clientes` en nodo04 muestra únicamente `norte` y `este`; en nodo05 únicamente `sur` y `oeste`.
- ☐ `SELECT COUNT(*) FROM pedidos` devuelve `10` en nodo04 y `10` en nodo05.
- ☐ `SELECT DISTINCT region FROM pedidos` en nodo04 muestra `norte` y `este`; en nodo05 muestra `sur` y `oeste`.
- ☐ `COUNT(detalle_pedidos nodo04) + COUNT(detalle_pedidos nodo05) = 36`.
- ☐ `SELECT COUNT(*) FROM productos` devuelve `10` en nodo04 y `10` en nodo05.
- ☐ Verificación de disjunción: `COUNT(*) FROM clientes WHERE region IN ('sur','oeste') = 0` en nodo04; `COUNT(*) FROM clientes WHERE region IN ('norte','este') = 0` en nodo05.
- ☐ Verificación de co-localización: `COUNT(*) FROM detalle_pedidos dp WHERE NOT EXISTS (SELECT 1 FROM pedidos p WHERE p.id = dp.pedido_id) = 0` en ambos shards.
- ☐ Verificación de integridad cliente-pedido: `COUNT(*) FROM pedidos p WHERE NOT EXISTS (SELECT 1 FROM clientes c WHERE c.id = p.cliente_id) = 0` en ambos shards.
- ☐ El JOIN local completo (D.10) en nodo04 devuelve solo regiones `norte` / `este`; en nodo05 solo `sur` / `oeste`.
- ☐ Las tres verificaciones de completitud en nodo01 (D.11) devuelven `OK`.
- ☐ `SHOW TABLES IN lab_bdd` en nodo01 no incluye `tmp_detalle_frag_A` ni `tmp_detalle_frag_B`.
- ☐ `SHOW GRANTS FOR 'shard_pull'@'192.168.56.104'` en nodo01 devuelve error o vacío (usuario eliminado).
- ☐ Los snapshots `fase13-completa` existen en los cinco nodos del laboratorio.
- ☐ Puedo explicar sin ver el documento la diferencia entre fragmentación primaria (`clientes`, `pedidos`) y derivada (`detalle_pedidos`).

- ☐ Puedo justificar por qué no hay FK en los shards y cómo la co-localización preserva la integridad referencial por diseño.
 - ☐ Puedo describir qué hará el coordinador nodo06 en la Fase 14 para ejecutar la consulta UNION ALL de forma transparente al cliente.
-

Preguntas teóricas para estudiantes

1. En la verificación de integridad cliente-pedido (D.10), se confirma que en nodo04 todos los pedidos tienen su cliente en el mismo nodo, y lo mismo en nodo05. Esto ocurre por diseño: ambas tablas se fragmentan con el mismo predicado (`region`). Describe un escenario de negocio realista en que esta garantía podría romperse — es decir, en que el cliente de un pedido estuviera en un shard diferente al pedido — y explica qué cambio en el proceso de inserción causaría esa situación. ¿Cómo detectaría el sistema ese problema sin una FK de motor?
2. La tabla `productos` se cargó completa en nodo04 y en nodo05, lo que implica que los 10 productos están almacenados dos veces en el sistema distribuido. Analiza el impacto de esta replicación temporal en: (a) el espacio de almacenamiento, (b) la consistencia cuando se actualiza el precio de un producto en nodo01 (¿se propaga automáticamente a nodo04 y nodo05?), y © las consultas `SELECT` sobre productos desde los shards. ¿Qué problema concreto resuelve la fragmentación vertical de la Fase 14 comparada con este estado actual?
3. La fragmentación derivada de `detalle_pedidos` se implementó mediante tablas temporales en nodo01 porque `detalle_pedidos` no tiene columna `region` . Proponer un diseño de esquema alternativo que agregue la columna `region` directamente a `detalle_pedidos` como columna derivada (desnormalización), y evaluar: (a) ventajas e inconvenientes de la desnormalización, (b) mecanismo para mantener `detalle_pedidos.region` consistente con `pedidos.region` sin FK cross-nodo, y © impacto en el tamaño de datos almacenados por shard.
4. Los shards tienen `log_bin = OFF` en su configuración. Explica las implicaciones de esta decisión en los siguientes escenarios: (a) recuperación ante fallo total de nodo04, (b) posibilidad de agregar un esclavo de lectura a nodo04 en fases futuras, y © auditoría de cambios en los datos del shard. ¿En qué circunstancia del laboratorio sería conveniente habilitar el binary log en los shards?

5. Un estudiante propone usar el snapshot `fase12-completa` (el más reciente) en lugar de `fase08-completa` como base para los clones, argumentando que incluye más configuraciones. Evalúa técnicamente esa propuesta: ¿qué configuraciones habría que eliminar o neutralizar en el clon?, ¿qué riesgo concreto existe si el archivo `60-replication-master.cnf` no se elimina del shard?, y ¿por qué `fase08-completa` es la base más limpia para un nodo shard autónomo?

Ejercicios prácticos

1. Script de auditoría de integridad referencial distribuida.

Escribir un script Bash en `nodo01` que, conectándose a `nodo04` y a `nodo05` vía `mysql -h ... -u shard_verify`, detecte automáticamente: (a) filas en `detalle_pedidos` de cada shard que no tienen su `pedido_id` en la tabla `pedidos` del mismo shard, y (b) filas en `pedidos` de cada shard que no tienen su `cliente_id` en `clientes`. El script debe imprimir `[NODO04] OK` o `[NODO04] FALLA: N huérfanos en detalle_pedidos` según el resultado. Insertar deliberadamente una fila huérfana en `nodo04`, confirmar que el script la detecta, eliminarla y corroborar que vuelve a `OK`.

2. Análisis del diseño alternativo con cuatro shards.

Si el laboratorio creciera a cuatro nodos de sharding (`nodo04` a `nodo07`), el diseño natural sería un shard por región (`norte`, `sur`, `este`, `oeste`). Documentar los predicados `WHERE` para esa fragmentación de cuatro fragmentos, calcular el número de filas esperadas por shard para `clientes`, `pedidos` y `detalle_pedidos`, y analizar si el diseño de 4 shards ofrece ventajas sobre el de 2 en términos de balanceo de carga, granularidad de poda en JOINS y coste de agregar una quinta región. No es necesario implementarlo.

3. Reconstrucción con el motor FEDERATEDX.

Desde `nodo01`, habilitar el motor FederatedX con `INSTALL SONAME 'ha_federatedx'`. Crear una tabla tipo FEDERATED en `nodo01` apuntando a la tabla `clientes` de `nodo04`:

SQL

```
CREATE TABLE clientes_shard_a (...estructura...)
ENGINE=FEDERATED
CONNECTION='mysql://shard_verify:ShardVerify_2025!@192.168.56.104/la
b_bdd/clientes';
```

Repetir para nodo05 (`clientes_shard_b`). Ejecutar una `UNION ALL` real entre las dos tablas FEDERATED y comparar el resultado con `lab_bdd.clientes` en nodo01. Documentar la salida y explicar por qué este enfoque con FEDERATED es el precursor conceptual directo de las tablas Spider que se usarán en la Fase 14.

Reto adicional para alumnos avanzados

Diseñar e implementar un **procedimiento de resincronización automatizada de shards** para el escenario en que nodo04 queda temporalmente desconectado de la red y, durante ese tiempo, se insertan en nodo01 nuevos pedidos para clientes de las regiones norte y este. Al reconectar nodo04, el shard está desactualizado.

El procedimiento debe: (1) identificar en nodo01 qué filas de `clientes`, `pedidos` y `detalle_pedidos` no están presentes en nodo04, comparando por `id` y por `fecha_alta` / `fecha_pedido`; (2) generar y ejecutar los `INSERT` diferenciales solo para las filas faltantes; (3) verificar que tras la sincronización los conteos coincidan y no existan duplicados. Documentar el mecanismo completo con comandos exactos y discutir sus limitaciones (¿qué ocurre si durante la desconexión se insertaron filas directamente en nodo04 que no están en nodo01?). Proponer qué funcionalidad de MariaDB — Galera Cluster, replicación semisíncrona u otra — podría automatizar este proceso en producción.

Criterios de evaluación para el profesor

Criterio	Peso	Indicador de logro
Creación y configuración de los nodos shard	20%	nodo04 y nodo05 existen con la IP, hostname y <code>server_id</code> correctos; MariaDB acepta conexiones remotas; las claves SSH fueron regeneradas; el estudiante puede listar los snapshots y describir la estrategia de clon enlazado
Esquema de shards sin FK	15%	<code>SHOW CREATE TABLE pedidos/detalle_pedidos</code> en ambos shards no muestra FK; el estudiante puede explicar por qué se eliminaron y cómo la co-localización las hace innecesarias en este diseño
Carga de fragmentos correcta	25%	Los conteos de clientes y pedidos son exactamente 10 en cada shard con las regiones correctas; la

		suma de detalles en ambos shards es 36; productos (10 filas) está en ambos shards
Verificación de correctitud (completitud, disjunción, co-localización)	20%	Las verificaciones SQL del bloque D.10 producen resultados OK en ambos shards; el estudiante puede ejecutar cada verificación y explicar qué mide; los JOINS locales funcionan sin acceder a datos de otro nodo
Reconstrucción y demostración global	10%	El estudiante puede mostrar que $\text{nodo04} + \text{nodo05} = \text{nodo01}$ para cada tabla fragmentada; puede escribir de memoria la consulta UNION ALL que el coordinador ejecutará en la Fase 14
Comprensión conceptual — preguntas teóricas	10%	Las respuestas distinguen fragmentación primaria y derivada, justifican la ausencia de FK

		con argumentos técnicos y relacionan el diseño con la Fase 14 usando términos como co-localización, predicado minterm, completitud y transparencia
--	--	--

I. Preparación para la siguiente fase

La **Fase 14: Fragmentación Vertical** requerirá:

- Los snapshots `fase13-completa` en los cinco nodos activos del laboratorio (completados en esta fase).
- `bdd-nodo04` y `bdd-nodo05` configurados como shards autónomos con su carga de datos verificada (esta fase).
- Comprensión del estado actual de `productos`: está completa y duplicada en nodo04 y nodo05 — la Fase 14 la eliminará de ambos y la reemplazará por los dos fragmentos verticales definidos en el DDD:
 - `V_productos_basico` (id, sku, nombre, categoria, precio, stock, fecha_creacion) → nodo04
 - `V_productos_detalle` (id, sku, descripcion, ficha_tecnica, imagen_url, peso_kg) → nodo05
- Creación de `bdd-nodo06` (192.168.56.106) al inicio de la Fase 14: será el nodo coordinador que implementa el motor **Spider** de MariaDB. Se creará como clon enlazado de `bdd-nodo01` desde el snapshot `fase08-completa`, con el mismo proceso de configuración aplicado en esta fase para nodo04/05.
- Habilitación del motor Spider en nodo06 con `INSTALL SONAME 'ha_spider';`, que permite crear tablas tipo SPIDER apuntando a tablas remotas en nodo04 y nodo05 e implementar la transparencia de fragmentación y ubicación ante el cliente.

No es necesario instalar ni configurar nada adicional en los nodos existentes. Los shards nodo04 y nodo05 ya tienen el esquema y los datos que la Fase 14 comenzará a reorganizar.
